

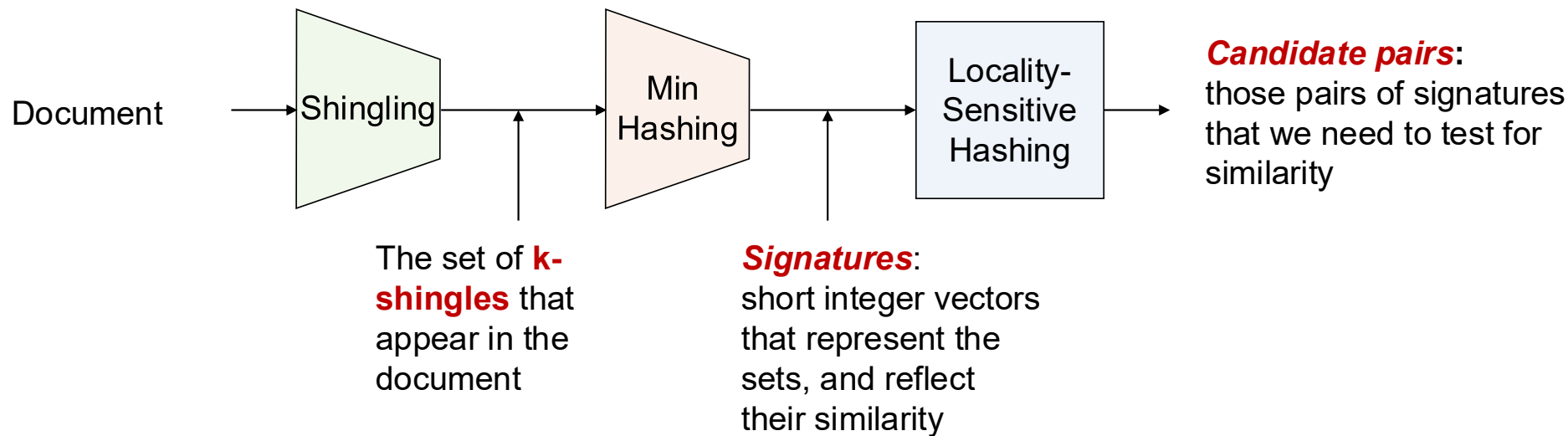
Locality-sensitive Hashing III

COMPSCI 753

Zijian Huang

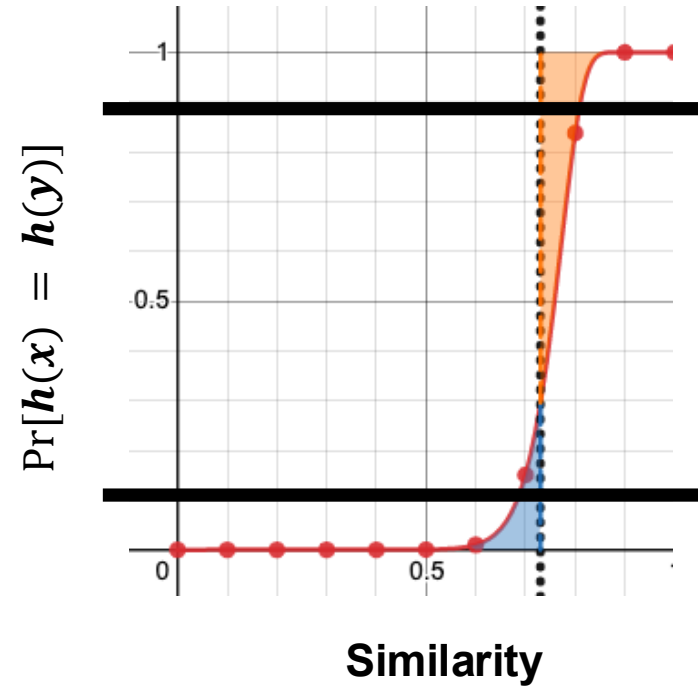
The University of Auckland

RECAP: Document Similarity Search by LSH



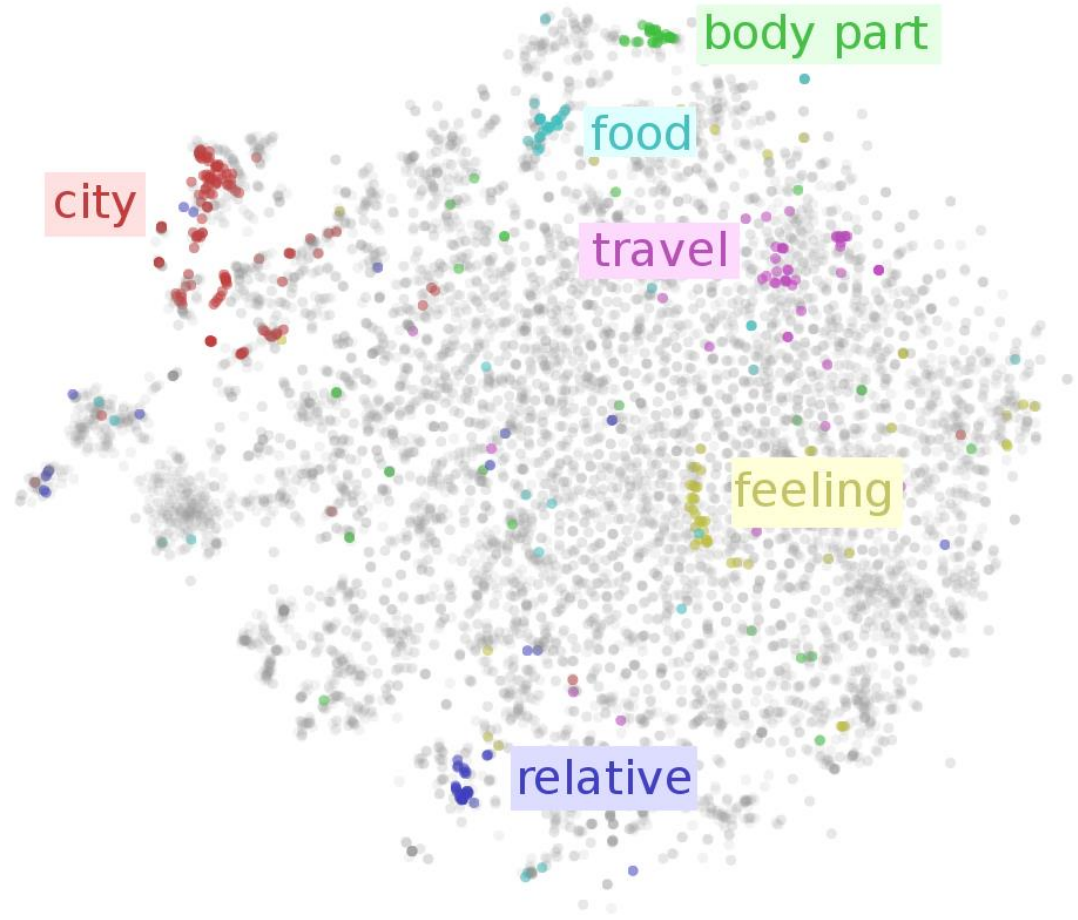
Outline

- Typical similarity/distance measure
 - Definitions
 - Applications
- Theory of locality-sensitive hashing
 - LSH definition
 - LSH framework for near neighbor search



Near Neighbors Search

Many real-world problems can be expressed as finding “similar” items or **find near-neighbors in high-dimensional space**



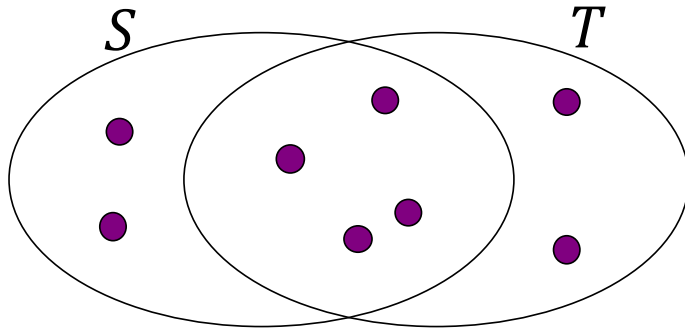
Source: <https://ruder.io/word-embeddings-1/>

Distance/Similarity Measure

- For each application, we need to define a similarity/distance measure to distinguish between similar (near) and dissimilar (far apart) objects
- Typical similarity/distance measures
 - Jaccard similarity/distance
 - Cosine similarity/distance
 - Hamming distance
 - Euclidean distance (L2)
 - Manhattan distance (L1)

Jaccard Similarity/Distance

- Jaccard similarity of two sets S and T : $J(S, T) = \frac{|S \cap T|}{|S \cup T|}$
- Jaccard distance of two sets S and T : $1 - J(S, T)$



- 4 in intersection
- 8 in union
- Jaccard similarity $J(S, T) = 1/2$
- Jaccard distance $1 - J(S, T) = 1/2$

Applications

- Texts/Documents:
 - Represent text/documents as a set of shingles
 - Applications: detect plagiarism, mirror pages, articles from the same source
- Recommender System:
 - Online-purchases: represent a user (item) as a set of items (users)
 - Netflix: represent a user (movie) as a set of rated movies (users)
- Core Technique: similarity computation between users-users, or items-items



Application: Purchasing Behavior Analyses

- Market basket data:
 - Represent an item as as et of transaction IDs
 - Association Rules: An implication expression of the form $X \rightarrow Y$ if $J(X,Y) \geq s$

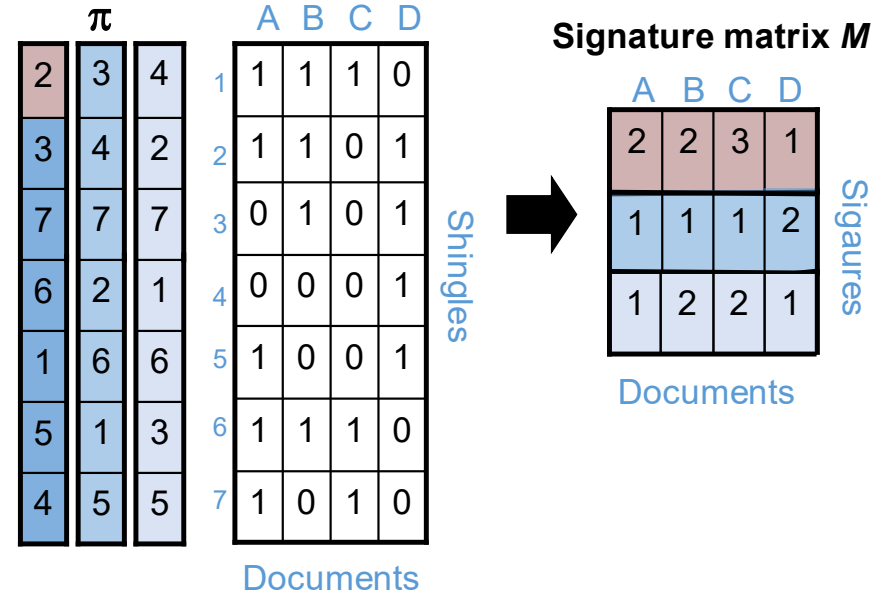
Market-Basket transactions

TID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

- Example of Frequent Itemset: {Diaper, Beer}
- Example of Association Rules: {Diaper} $\rightarrow$ {Beer}
 - Diapers = {2, 3, 4, 5}
 - Beer = {2, 3, 4}
 - $J(\text{Diapers}, \text{Beer}) = 3/4$

Application: MinHash Signature (Broder'97)

- Permute the Boolean matrix with a random permutation π
- MinHash function $h_{\pi}(S)$
 - The index of the first row with value 1 of S in permuted order π
- Use several permutation π to construct the signature of each document



Python Code:

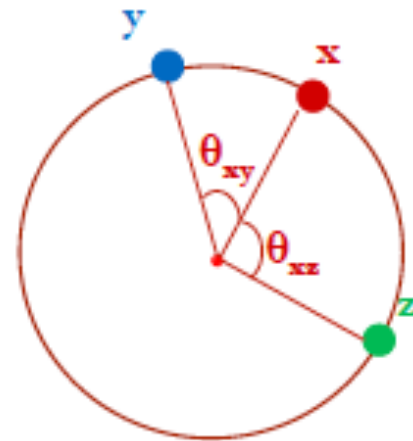
https://www.bogotobogo.com/Algorithms/minHash_Jaccard_Similarity_Locality_sensitive_hashing_LSH.php

Cosine Similarity/Distance

- Given two points $\mathbf{x} = (x_1, \dots, x_d)$ and $\mathbf{y} = (y_1, \dots, y_d)$ in high-dimensional sphere with $\|\mathbf{x}\|_2^2 = \|\mathbf{y}\|_2^2 = 1$, where

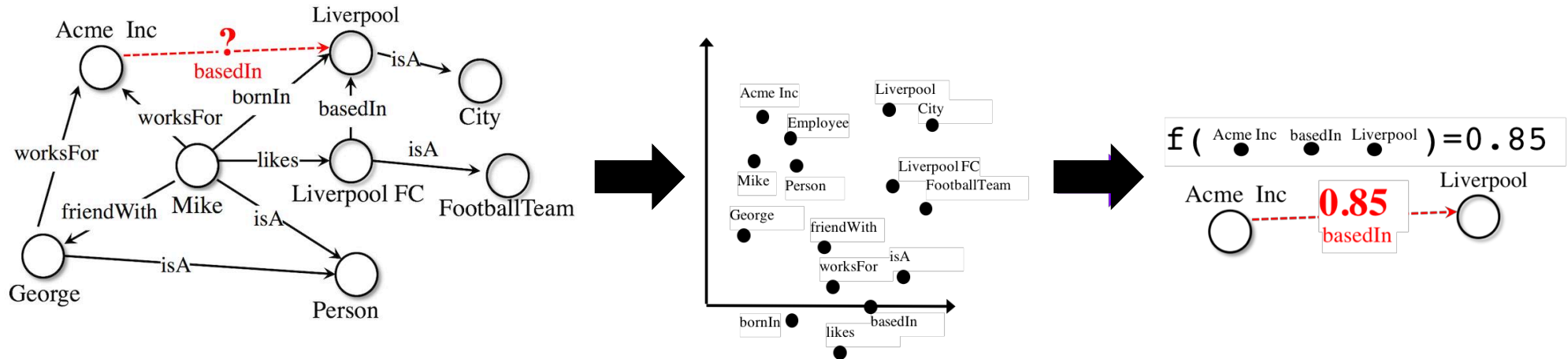
$$\|\mathbf{x}\|_2^2 = \sqrt{x_1^2 + x_2^2 + \dots + x_d^2} = 1$$

- Cosine similarity: $\cos(\theta_{xy}) = \sum_{i=1}^d x_i y_i$, where $0 \leq \theta_{xy} \leq \pi$
- Cosine distance: $1 - \cos(\theta_{xy})$



Applications: Knowledge Graph Embedding

- Learn the low-dimensional vector for each entity and relation in knowledge graphs (KGs), e.g., graph neural network (GNNs), translational models



Source: <https://github.com/Accenture/AmpliGraph/>

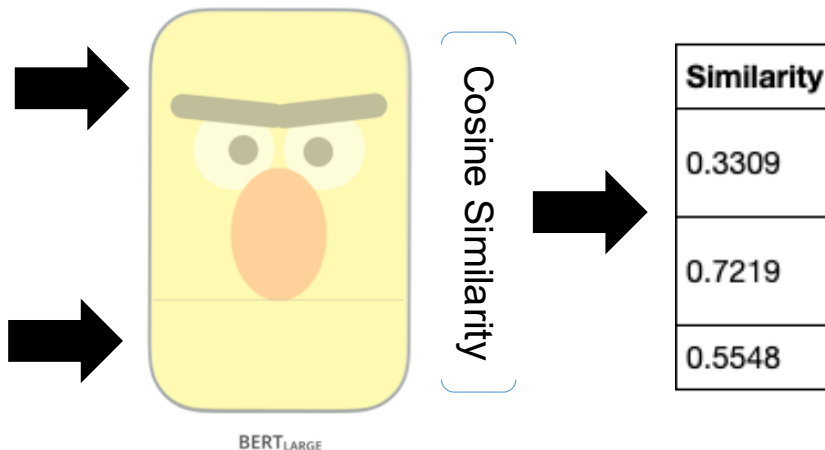
Application: Document Similarity in NLP

- Learn the low-dimensional vector for each sentence or document in textual corpus, e.g., language models (BERT, GPT-3, RoBERTa etc.)

Query: "Three years later, the coffin was still full of Jello"

Index	Sentence
1	"The fish dreamed of escaping the fishbowl and into the toilet where he saw his friend go."
2	"The person box was packed with jelly many dozens of months later."
3	"He found a leprechaun in his walnut shell."

...



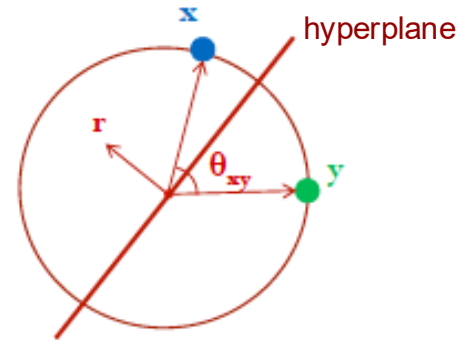
Application: Near Duplicate document (SimHash)

- Each document is represented as a set of weighted features (i.e., a vector).
- **SimHash** [Charikar'02] is for duplicate document detection which can quickly estimate how similar two document vectors are.
- Generate a random Gaussian vector $\mathbf{r} = (r_1, \dots, r_d)$ where r_i is drawn from $N(0, 1)$

• SimHash function:

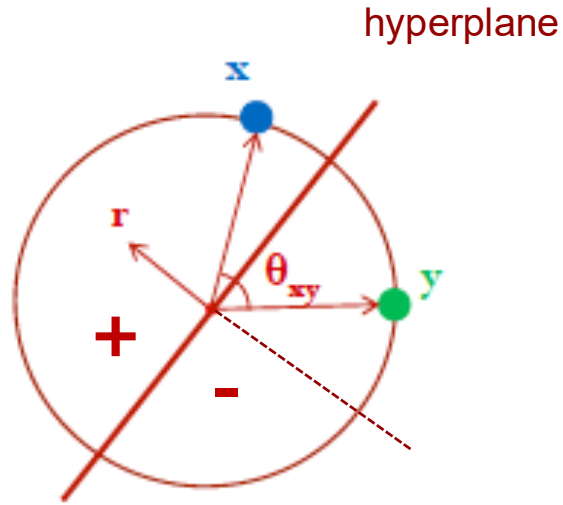
$$h_r(x) = \begin{cases} 1 & \text{if } \cos(\theta_{xr}) \geq 0 \\ 0 & \text{if } \cos(\theta_{xr}) < 0 \end{cases}$$

$$Pr[h_r(x) = h_r(y)] = 1 - \theta_{xy}/\pi$$



Python Code:

https://www.bogotobogo.com/Algorithms/Locality_Sensitive_Hashing_LSH_using_Cosine_Distance_Similarity.php



Observation: only planes that resides between the two points will give the two vectors different SimHash values

$$\begin{aligned} \cos(\theta_{xr}) &> 0 \\ \cos(\theta_{yr}) &< 0 \end{aligned} \quad h_r(x) = \begin{cases} 1 & \text{if } \cos(\theta_{xr}) \geq 0 \\ 0 & \text{if } \cos(\theta_{xr}) < 0 \end{cases}$$

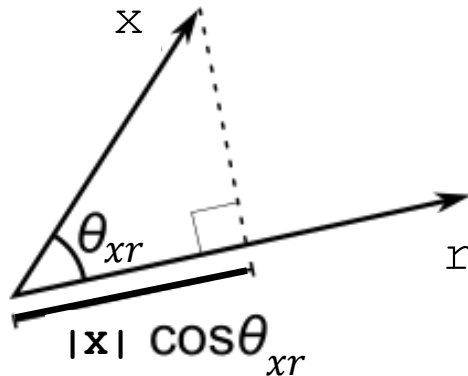
→ Prob. that planes reside between two points

$$\theta_{xy}/\pi$$

→ Two points with tiny angular difference to collide;

→ Two points with large angular difference to be placed in different bucket

$$\Pr[h_r(x) = h_r(y)] = 1 - \theta_{xy}/\pi$$

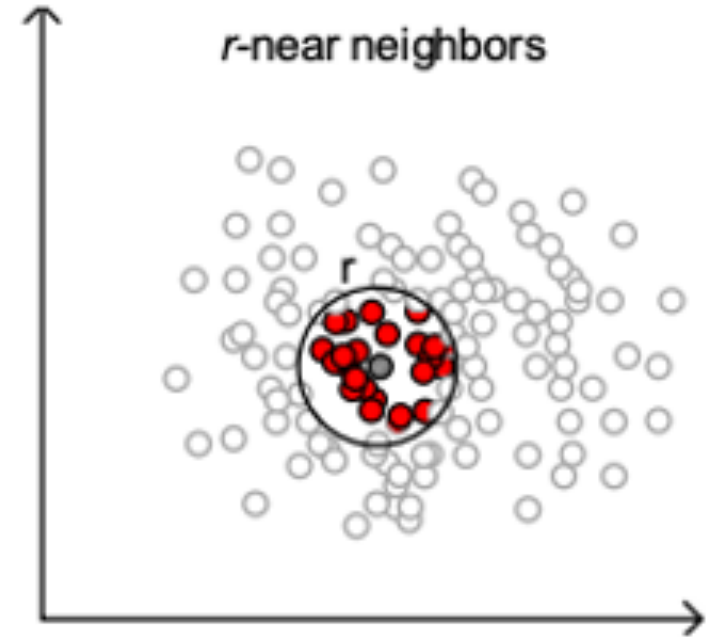


→ Uniformly randomly generate multiple vectors $h_1, h_2, \dots, h_k$, each point is represented as a k -dimensional SimHash signature

x	1	0	...	0	1
y	1	1	...	0	1

Outline

- Popular similarity/distance measure
 - Definitions
 - Applications
- **Theory of locality-sensitive hashing**
 - LSH definition
 - LSH framework for near neighbor search

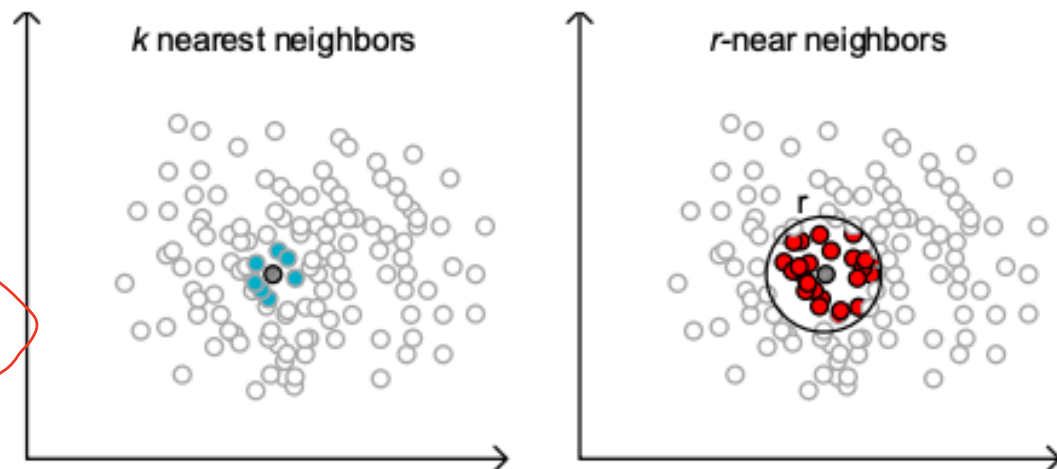


r-Near Neighbor Search (rNNS) Problem

- Given a data set $S \subset R^d$, a distance function $\mathbf{d}(x, y)$, a distance threshold r , and a query $q \in R^d$, return some point $x \in S$ where $\mathbf{d}(x, q) \leq r$

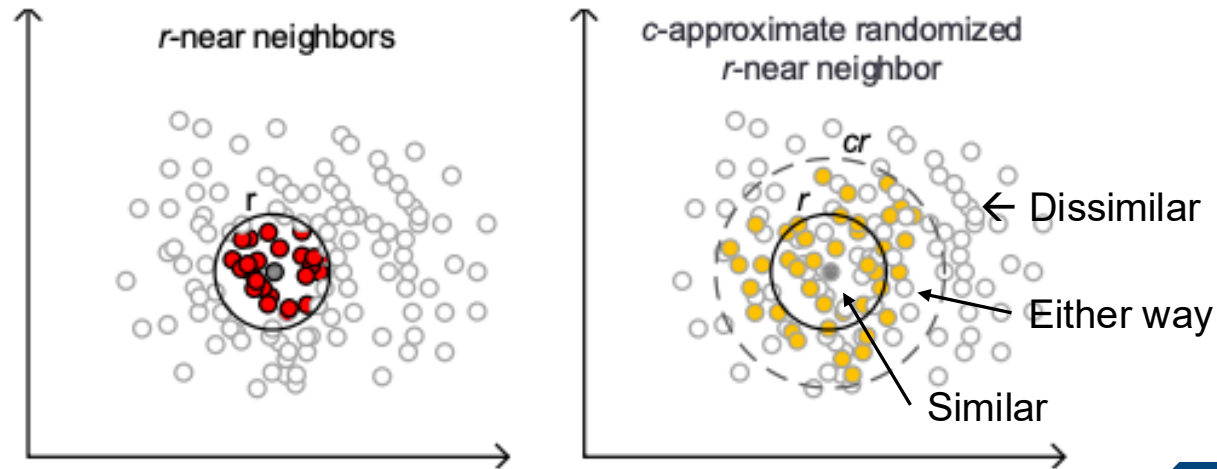
- Other variant
 - k -nearest neighbors search

固定半径 r



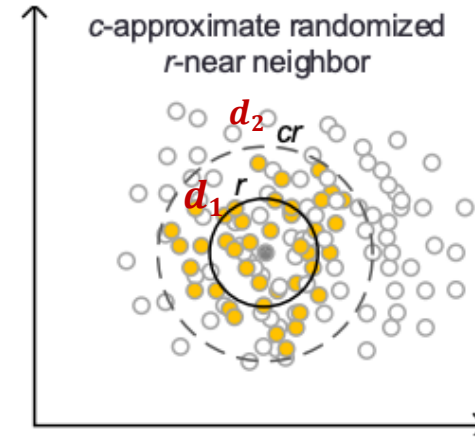
c -approximation Randomized rNNS

- Given a data set $S \subset R^d$, a distance function $d(x, y)$, parameters $r > 0, c > 1, d > 0$, and a query $q \in R^d$, with **probability $1 - \delta$**
 - If $x \in S$ and $d(x, q) \leq r$ exists, return some point $x' \in S$ where $d(x', q) \leq cr$
 - Else, return nothing



LSH Definition[Indyk & Motwani'98]

- Given a distance function $d: U \times U \rightarrow \mathbf{R}$ and positive values r, c, p_1, p_2 , where $p_1 > p_2, c > 1$
- A family of functions H is called (d_1, d_2, p_1, p_2) -sensitive if for uniformly chosen $h \in H$ and all $x, y \in U$
 - If $d(x, y) \leq r \rightarrow \Pr[h(x) = h(y)] \geq p_1$ (similar/near points)
 - If $d(x, y) \geq cr \rightarrow \Pr[h(x) = h(y)] \leq p_2$ (dissimilar/far away points)

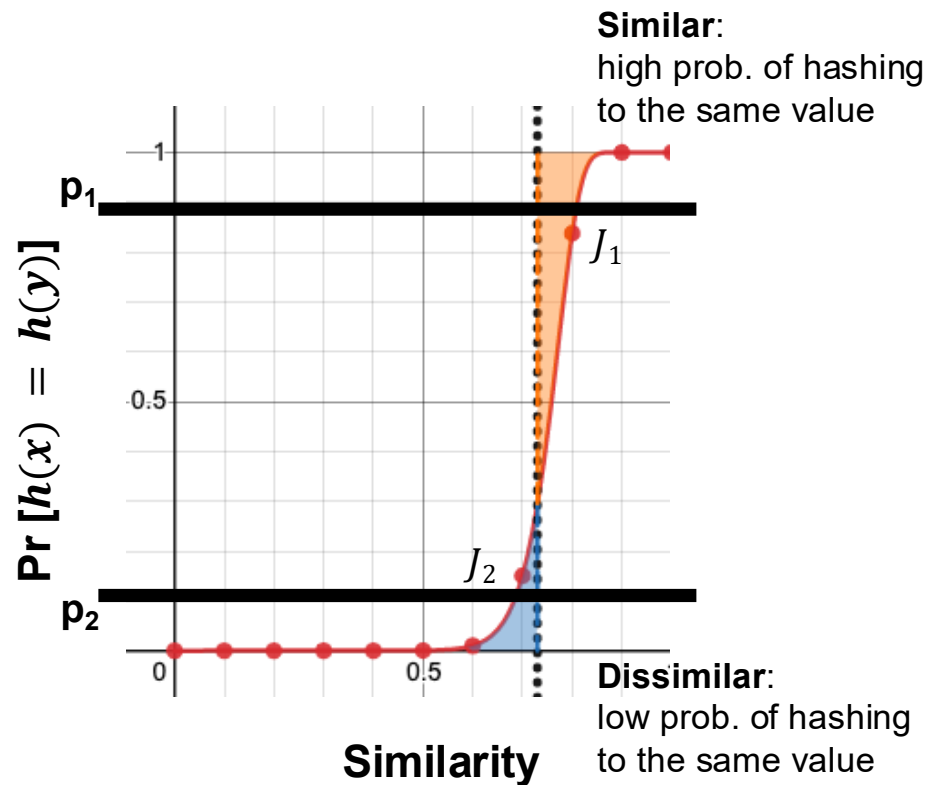


How to make p_1 as large as possible?

How to make p_2 as small as possible (i.e., $1/n$) ?

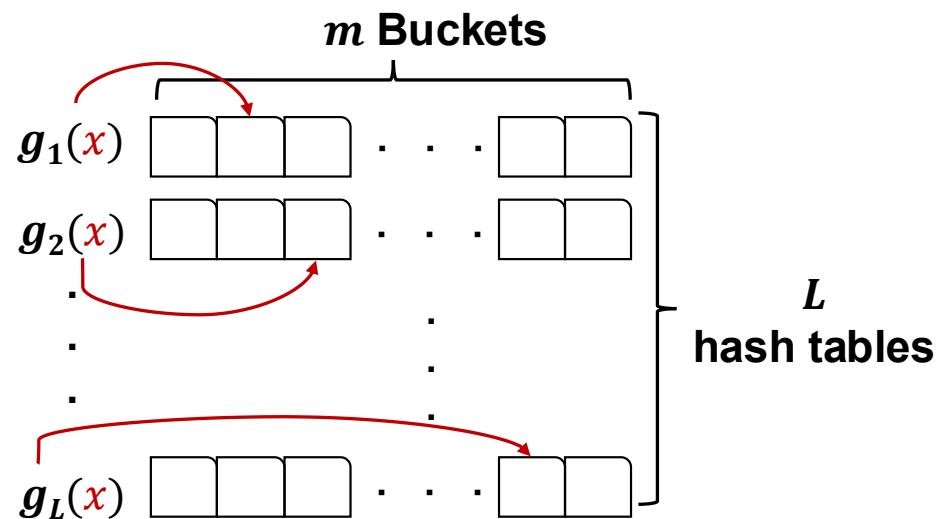
Example: LSH Family

- MinHash is locality-sensitive hashing
 - $\Pr_{\pi}[h_{\pi}(S) = h_{\pi}(T)] = J(S, T)$
 - $((1 - J_1), (1 - J_2), p_1, p_2)$ -sensitive, where $J_1 > J_2$
 - What are the values of p_1, p_2 given r and cr ?
- SimHash is locality-sensitive hashing
 - $\Pr_r[h_r(x) = h_r(y)] = 1 - \theta_{xy} / \pi$
 - $(\theta_1, \theta_2, p_1, p_2)$ -sensitive, where $\theta_1 < \theta_2$
 - What are the values of p_1, p_2 given r and cr ?



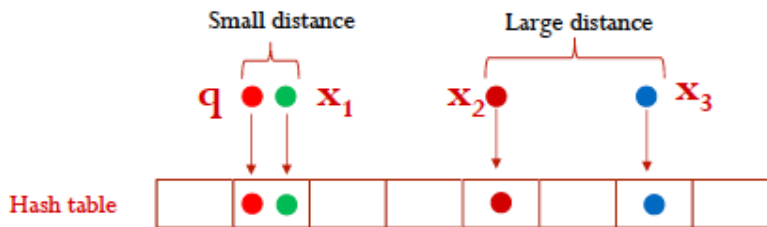
Outline

- Typical similarity/distance measure
 - Definitions
 - Applications
- Theory: c-approximation Randomized rNNS
 - General setting
 - Hash table construction
 - Hash table lookup
 - Performance guarantee
 - Correctness



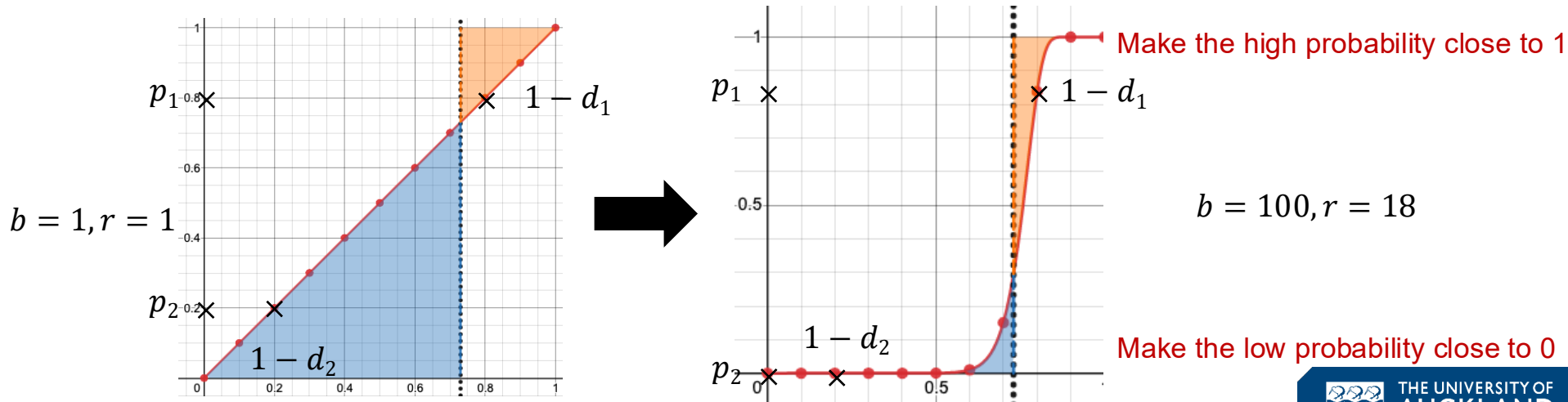
LSH Framework

- Brute-force algorithm for rNNS: $O(n)$
- Claim
 - We can solve the randomized c -approximation r -near neighbor search in sublinear time using locality-sensitive hashing
 - **Sublinear** time: $O(n^\rho)$, $0 < \rho < 1$ for any query and data set of size n
- Intuition: **Candidate** points are points within the same bucket of the **query** point



General Settings to Amplify LSH Family

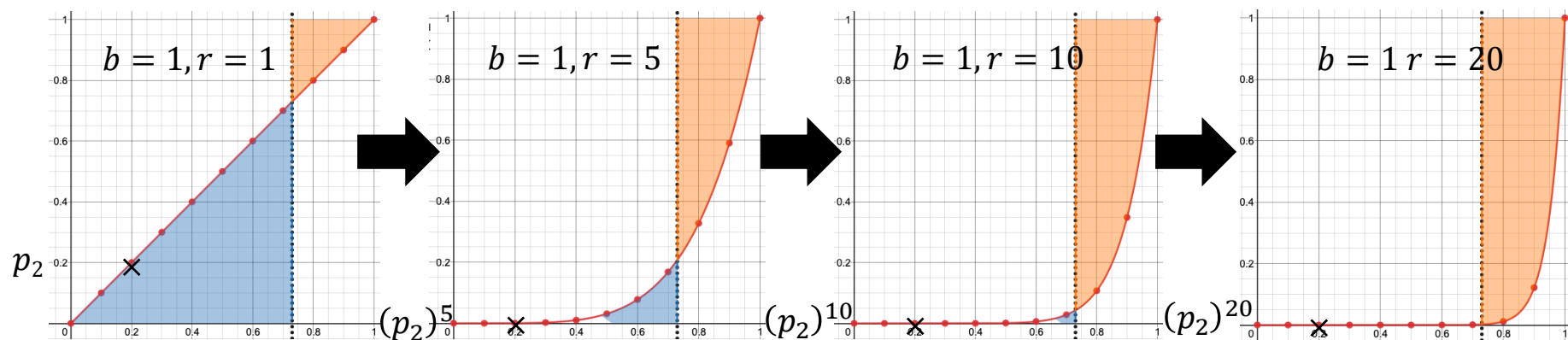
- Given any (d_1, d_2, p_1, p_2) -sensitive family, how to amplify the gap between p_1 and p_2
 - Hash function concatenation (e.g., “rows in a band”): k ← mirrors the effect of combining several rows
 - Hash table construction (e.g., “many bands”): L ← mirrors the effect of combining several bands
- E.g., MinHashing (b, r) to find best S-curve



Concatenation of Multiple Hash Functions (k)

- Goal: make the small probability p_2 get very close to 0, i.e., find $(d_1, d_2, (p_1)^k, (p_2)^k)$ -sensitive family
- Define a new hash function $\mathbf{g}(x) = [h_1(x), h_2(x), \dots, h_k(x)]$ where $h_i, 1 \leq i \leq k$, are chosen independently at random from a LSH family $\mathbf{H}$
 - If $d(x, y) \leq r \rightarrow \Pr[\mathbf{g}(x) = \mathbf{g}(y)] \geq (p_1)^k$ (similar/near points)
 - If $d(x, y) \geq cr \rightarrow \Pr[\mathbf{g}(x) = \mathbf{g}(y)] \leq (p_2)^k$ (dissimilar/far away points)

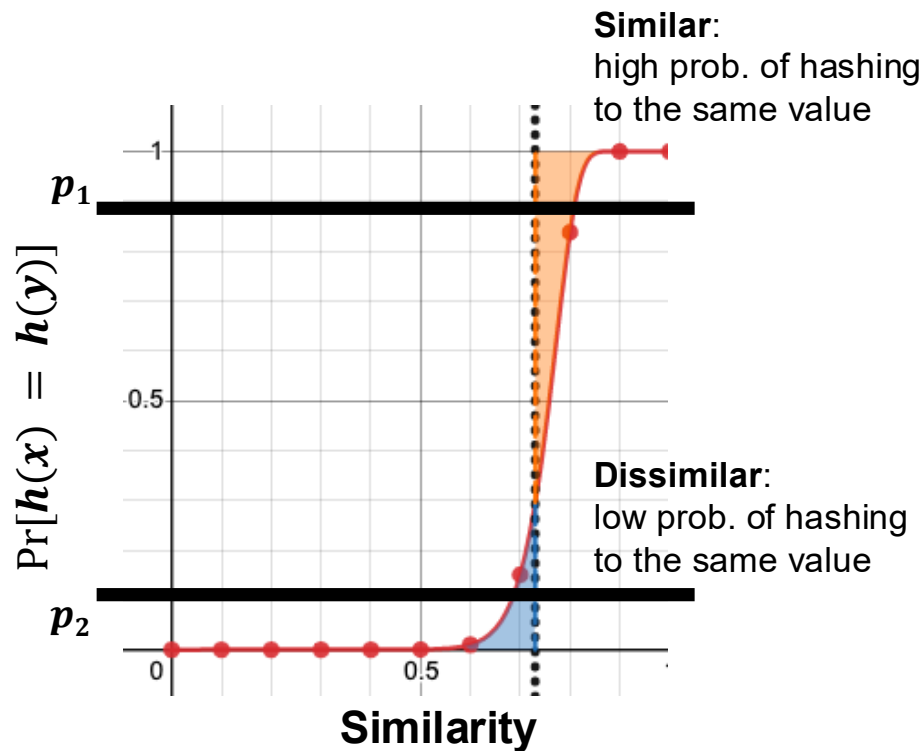
Lowers probability for large distances (Good)
i.e., corresponds to creating a band of size r



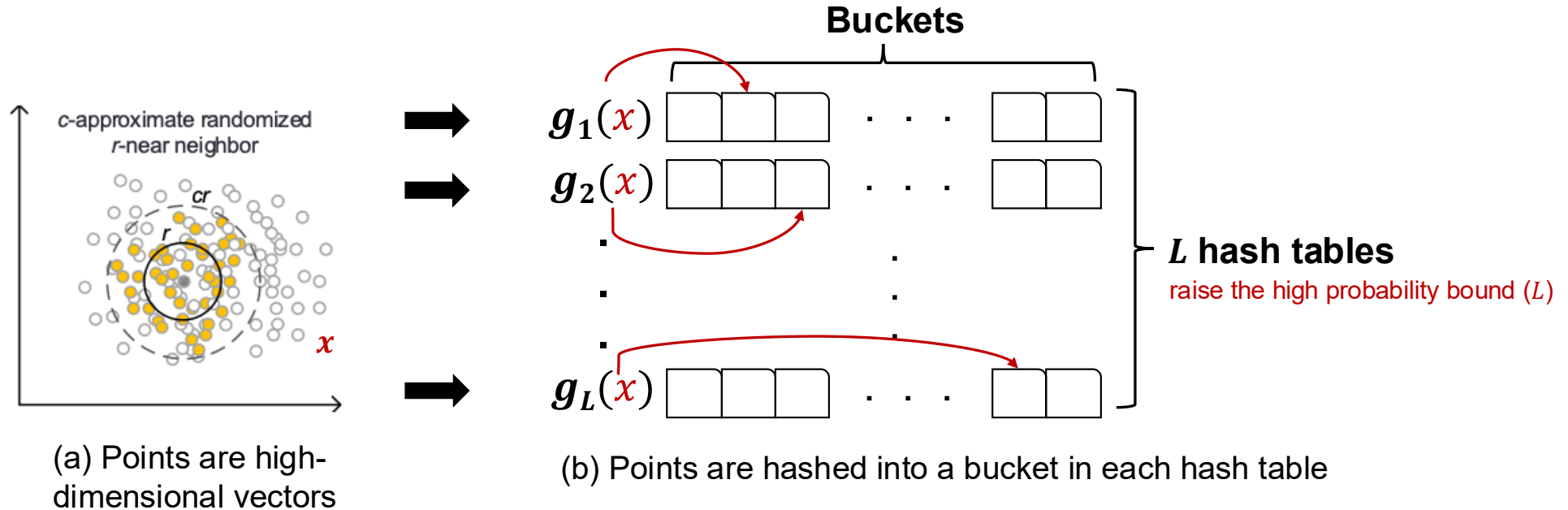
Concatenation of Multiple Hash Functions (k)

- If we want $(p_2)^k \leq 1/n$
- Then, set $k = \log(n) / \log(1/p_2)$, to achieve the following performance guarantee
 - $(p_2)^k = p_2^{\log n / \log(1/p_2)} = 1/n$
 - $(p_1)^k = p_1^{\log n / \log(1/p_2)} = 1/n^\rho$
 - where $0 < \rho = \frac{\log(1/p_1)}{\log(1/p_2)} < 1$

Small collision between q and dissimilar points (Good)



LSH Framework: Hash Tables Construction (L)



$$g(x) = [h_1(x), h_2(x), \dots, h_k(x)], \text{ where } h_i \in H$$

- Prob. collision of near points: $1/n^p$
- Prob. collision of far way points: $1/n$

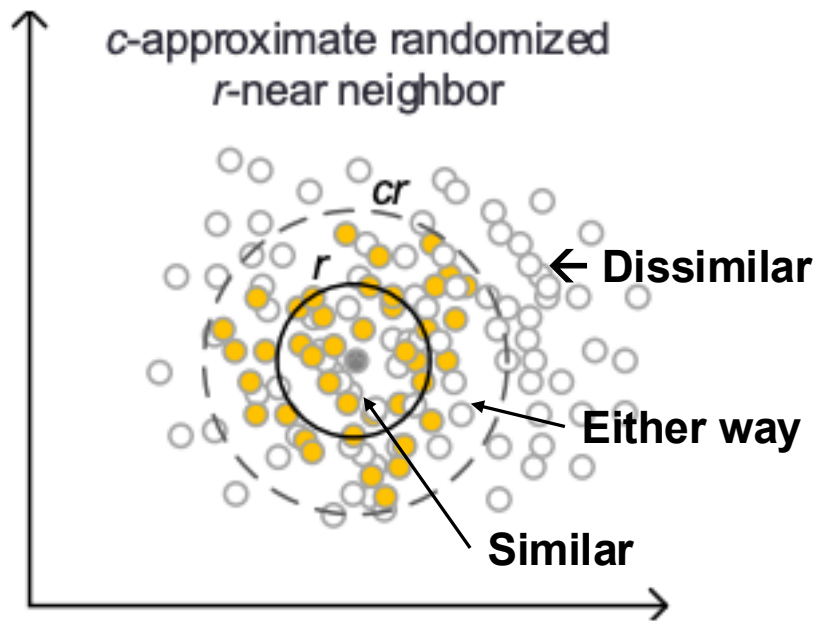
lower the low probability bound (k)

LSH Framework Intuition

- Similar points:
 - If $d(x, y) \leq r$ then $\Pr[g(x) = g(y)] \geq 1/n^p$
 - If there is a similar point, i.e., $d(x, q) \leq r$, in expectation, we need to check n^p points to find a similar point in the same bucket of query



- Dissimilar points:
 - If $d(x, y) \geq cr$ then $\Pr[g(x) = g(y)] \leq 1/n$
 - Since we have n points, in expectation, only 1 dissimilar point is in the same bucket of query



LSH Framework Intuition (L)

- Goal: make the larger probability approach to 1, find
- $(d_1, d_2, 1 - (1 - p_1)^L, 1 - (1 - p_2)^L)$ -sensitive family
- Similar points:
 - If $d(x, y) \leq r$ then $\Pr[g(x) = g(y)] \geq 1/n^\rho$
 - If there is a similar point, i.e., $d(x, q) \leq r$, in expectation, we need to check n^ρ points to find a similar point in the same bucket of query
- Dissimilar points:
 - If $d(x, y) \geq cr$ then $\Pr[g(x) = g(y)] \leq 1/n$
 - Since we have n points, in expectation, only 1 dissimilar point is in the same bucket of query

Let $L = n^\rho$ hash tables

In expectation, less than $L = n^\rho$ points

E.g., $(d_1 = 0.2, d_2 = 0.6, p_1 = 0.8, p_2 = 0.4)$ –sensitive,
 $k = 4, L = 4$

- Family Transformation: (d_1, d_2, p_1, p_2) -sensitive $\rightarrow (d_1, d_2, 1 - (1 - p_1^k)^L, 1 - (1 - p_2^k)^L)$ -sensitive
 - Construct $k = 4$ MinHash functions to make the small probability approach to 0
 - Construct $L = 4$ hash tables to make the larger probability approach to 1

p	$1 - (1 - p^4)^4$
0.2	0.0064
0.3	0.0320
p_2 0.4	0.0985
0.5	0.2275
0.6	0.4260
0.7	0.6666
p_1 0.8	0.8785
0.9	0.9860

Original LSH: $(d_1 = 0.2, d_2 = 0.6, p_1 = 0.8, p_2 = 0.4)$ -sensitive

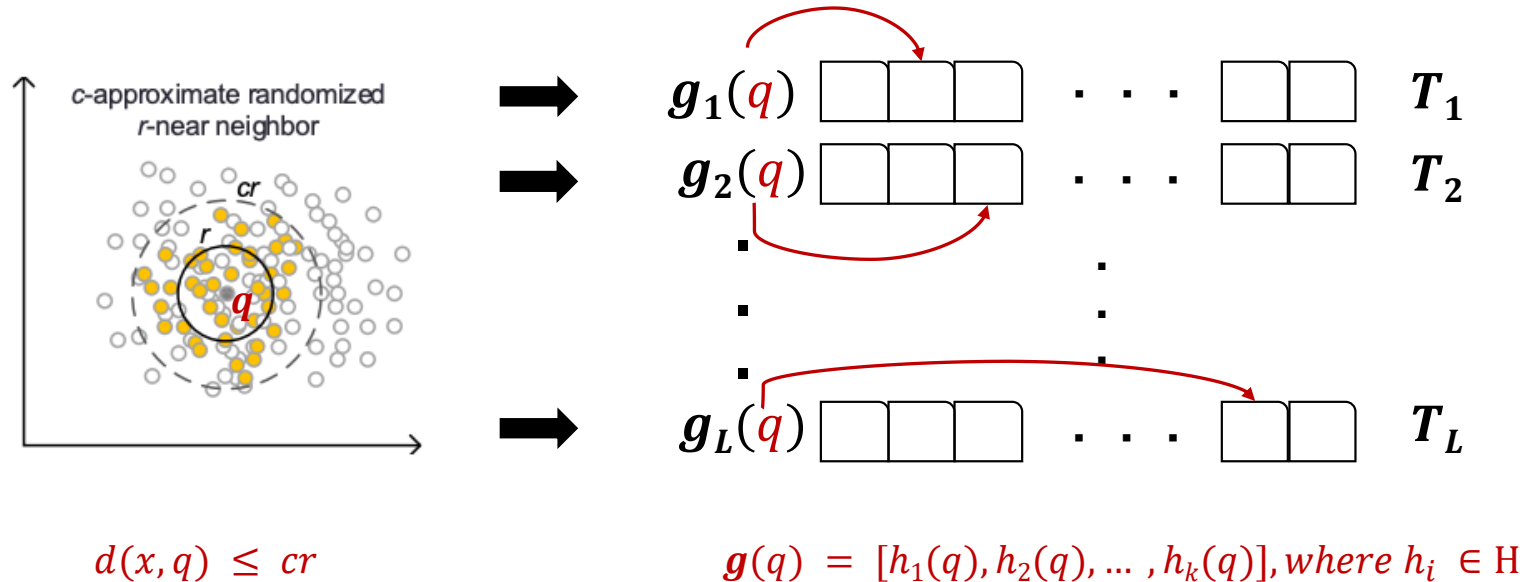
Transformed LSH: $(d_1 = 0.2, d_2 = 0.6, p_1 = 0.8785, p_2 = 0.0985)$ -sensitive

LSH Framework: Hash Tables Construction (L)

- Preprocessing
 - Choose L hash function $g_j, 1 \leq j \leq L$, by setting $g_j = (h_{1,j}, h_{2,j}, \dots, h_{k,j})$ where $h_{i,j}, 1 \leq i \leq k$, are chosen independently and randomly from the LSH family H
 - Construct L hash tables, where each hash table $T_j, 1 \leq j \leq L$, contains the n data points hashed using the hash function g_j
- Complexity Analysis
 - Preprocessing time: $O(Lkdn)$
 - d is the time to evaluate a function $h_i \in H$ on an input point x
 - Space: $O(nL) + O(kLn)$, storing data points

LSH Framework: Hash Tables Lookup

- Idea: We interrupt search after finding first $L' = 2 n^\rho$ points, including duplicates



LSH Framework: Hash Tables Lookup

- Query for a point q
 - For each $j = 1, 2, \dots, L$
 - Retrieve the points colliding with q from the bucket $g_j(q)$ in the T_j hash table
 - For each of the retrieved point x
 - Compute the distance $d(x, q)$
 - If $d(x, q) \leq cr$, return x and stop
 - Stop as soon as the number of retrieved points is more than L'
- Complexity Analysis
 - Query time: $O(dkL + d'L')$ with the assumption that d is the time to evaluate a function $h_i \in H$ for a point, d' is the time for evaluating the actual distance.

Compute hash values

Compute actual distance with L' points

Complexity Analysis

- Parameter settings:

$$k = \log n / \log(1/p_2)$$

$$0 < \rho = \frac{\log(1/p_1)}{\log(1/p_2)} < 1$$

$$L = n^\rho \text{ hash tables}$$

$$L' = 2 n^\rho \text{ points for computing actual distance}$$



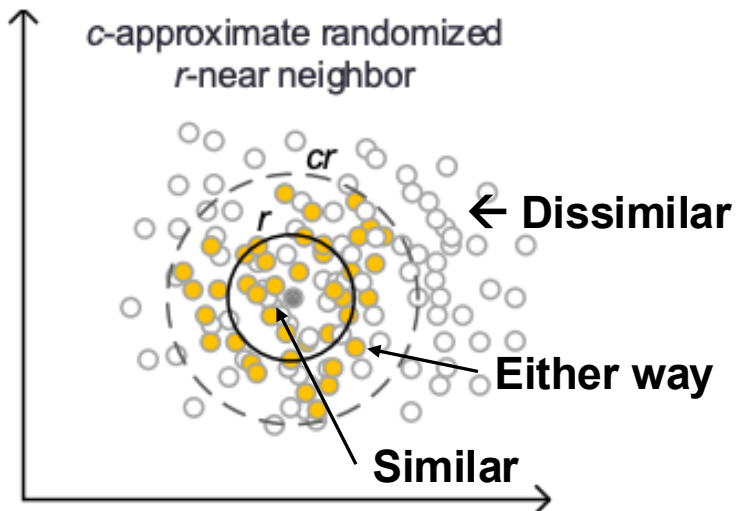
$g(x) = [h_1(x), h_2(x), \dots, h_k(x)]$, where $h_i \in H$

- Prob. collision of near points: $1/n^\rho$
- Prob. collision of far way points: $1/n$

- Preprocessing
 - Space: $O(nL + kLn)$
 - Time: $O(dkn^{1+\rho})$ with the assumption that d is the time to evaluate a function $h_i \in H$ for a point
- Query
 - Sublinear time: $O(dkn^\rho + 2d'n^\rho)$ with the assumption that d is the time to evaluate a function $h_i \in H$ for a point, d' is the time for evaluating the actual distance.

RECAP: c –approximation Randomized rNNS

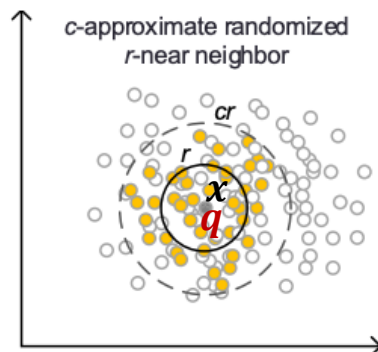
- Given a data set $S \subset R^d$, a distance function $\mathbf{d}(x, y)$, parameters
- $r > 0, c > 1, d > 0$, and a query $\mathbf{q} \in R^d$, with **successful probability $1 - \delta$**
 - If exists $x \in S$ and $\mathbf{d}(x, q) \leq r$, return some point $x' \in S$ where $\mathbf{d}(x', q) \leq cr$
 - Else, return nothing



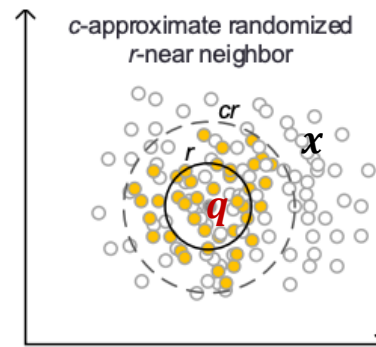
Correctness: Two Cases of Results

- We need to show that, with successful probability $1 - \delta$
 - **Case 1 (we can find x):** If exists $x \in S$ and $d(x, q) \leq r$, there exists a hash table T_j such that $g_j(x) = g_j(q)$ for some $1 \leq j \leq L$
 - **Case 2 (we find nothing):** The total number of collisions from far away points is at most $2L$. This means that after checking $2L$ points, we are certain that all points are far away from q more than cr

Case 1 (We can find x)



Case 2 (We find nothing)



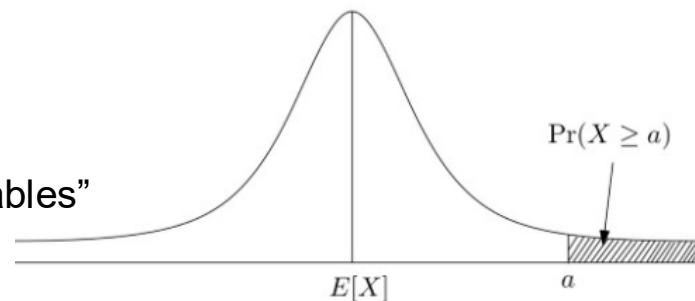
Case 1: We can find x

- **Observation:** If $d(x, q) \leq r$, then
 - Prob. x colliding with q in a hash table: $\geq 1/n^\rho$
 - Prob. x not colliding with q in a hash table: $\leq 1 - 1/n^\rho$
 - Prob. x not colliding with q in all L hash tables: $\leq (1 - 1/n^\rho)^L$ ← $(1 - 1/x)^x \approx 1/e$,
prob. of failure
 - Prob. x not colliding with q in all L hash tables: $1/e$ with $L = n^\rho$
 - Prob. x colliding with q in at least 1 hash tables: $1 - 1/e$ with $L = n^\rho$
- With prob. $1 - 1/e$, we can successfully find x if there is any near point x will collide with q in at least 1 hash table

Case 2: Return Nothing

- Observation:
 - Maximum number of far away points: n
 - If $d(x, q) \geq cr$, then $\Pr[h(x) = h(q)] \leq 1/n$
 - In expectation, we have at most 1 far away point collide with q in 1 hash table
 - $E[\text{number of far away points collided with } q \text{ in } L \text{ tables}] \leq L$

- Using Markov's inequality
 - Let X be the “number of far away points collided with q in L tables”
 - $\Pr[X \geq 2L] \leq \frac{E[X]}{2L} \leq 1/2$



Prob. of the shaded region is bounded, i.e., $\Pr[X \geq a] \leq \frac{E[X]}{a}$

- With prob. 1/2, we are certain that all points are far away after checking all first $L' = 2L$ points far away from q . Hence, return nothing

Correctness Summary

- **Successful Rate:** the probability that both case 1 and case 2 hold is at least:

$$1 - \underbrace{((1 - 1/2))}_{\text{Pro. case 2 fails}} + \underbrace{(1 - (1 - 1/e))}_{\text{Pro. case 1 fails}} = 1/2 - 1/e$$

Pro. case 2 fails

Pro. case 1 fails

- **Theorem:** There exists a sublinear algorithm to answer the c-approximate rNNS with probability $1/2 - 1/e$
- **Boosting the accuracy:** by repeating the LSH algorithm $O(\log(\frac{1}{\delta}))$ times, we can amplify the probability of success in at least 1 trial to $1 - \delta$ for any $\delta > 0$

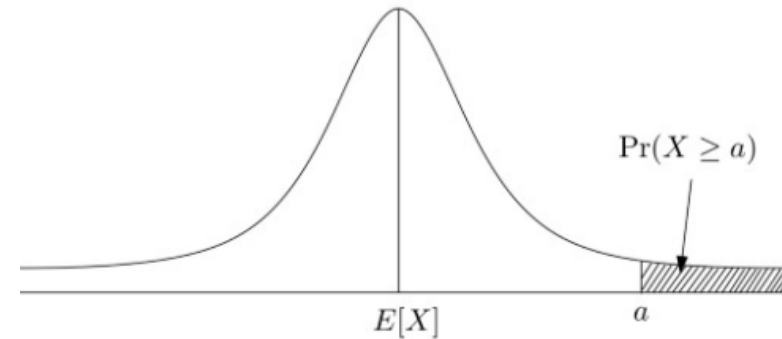
Example: Boosting Accuracy c -approximation rNNS

- **Idea:** Given an event e with success probability p , we run the algorithm t times for the same query but with different LSH indexes
- **Goal:** successful probability in at least 1 trial is $1 - \delta$ for any $\delta > 0$
 - Prob. e is successful $\geq p$
 - Prob. e is not successful $\leq 1 - p$
 - Prob. e is not successful on all t trials $\leq (1 - p)^t$
 - Prob. e is successful in at least 1 trial $\geq 1 - (1 - p)^t$
 - Choosing $(1 - p)^t = \delta$, then $t = O(\log(1/\delta))$

← Repeating the event t times

Summary

- Problem: r-Near Neighbor Search (rNNS)
- Relaxed Problem: c-approximation
Randomized rNNS
- Theory of LSH for c-approximation
Randomized rNNS
 - General setting
 - Hash table construction
 - Hash table lookup
 - Performance guarantee
 - Correctness



Prob. Of the shaded region is bounded, i.e., $\Pr[X \geq a] \leq \frac{E[X]}{a}$

Resources

- Python Code Example
 - MinHash Signature Generation:
https://www.bogotobogo.com/Algorithms/minHash_Jaccard_Similarity_Locality_sensitive_hashing_LSH.php
 - SimHash Signature Generation:
https://www.bogotobogo.com/Algorithms/Locality_Sensitive_Hashing_LSH_using_Cosine_Distance_Similarity.php
- Book
 - Algorithm Design by Jon Kleinberg and Éva Tardos. Chapter 13.
<https://www.cs.princeton.edu/~wayne/kleinberg-tardos/>
 - Mining of Massive Datasets by Jure Leskovec, Anand Rajaraman, and Jeff Ullman. Chapter 3.6 <http://mmds.org>